

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)

Assessment Tool Weightage: 50%

Dr G.A. SENTHIL

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.
(10 Marks)
2. Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.
(10 Marks)
3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.
(10 Marks)
4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.
(10 Marks)
5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of

deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ϵ -greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

Code of Conduct:

I POLIDAS BHARATH____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: bharath

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Grid World Navigation Using Reinforcement Learning

Aim

Develop a Python-based Reinforcement Learning model using a Grid World environment. The agent must reach the goal while avoiding obstacles.

Problem Description

Consider a 5×5 **Grid World**. The agent starts from the top-left corner and must reach the goal at the bottom-right corner. Some cells are obstacles, and the agent cannot move through them.

RL Components

1. State Space

The state represents the current position of the agent.

For a 5×5 grid:

$$S = \{(0, 0), (0, 1), \dots, (4, 4)\}$$

There are 25 possible states.

2. Action Space

The agent can perform four actions:

- 0 $\rightarrow$ Up
- 1 $\rightarrow$ Down
- 2 $\rightarrow$ Left
- 3 $\rightarrow$ Right

3. Reward Function

Situation	Reward
Reach goal	+10
Move to normal cell	-1
Hit obstacle	-5
Try to move outside grid	-5

The reward encourages the agent to reach the goal using a short and valid path.

4. Constraints

- The agent cannot move outside the grid.
- The agent cannot enter obstacle cells.
- The agent must eventually reach the goal.
- The number of steps can be limited to avoid infinite episodes.

Python Implementation

```
import gym
from gym import spaces
import numpy as np

class GridWorldEnv(gym.Env):

    def __init__(self):
        self.grid_size = 5

        # Start and goal
        self.start = (0, 0)
        self.goal = (4, 4)

        # Obstacles
        self.obstacles = [(1, 1), (2, 2), (3, 3)]

        # 25 possible states
        self.observation_space = spaces.Discrete(25)

        # 4 actions: Up, Down, Left, Right
        self.action_space = spaces.Discrete(4)
```

```

self.state = self.start

def reset(self):
    self.state = self.start
    return self.state[0] * self.grid_size + self.state[1]

def step(self, action):

    row, col = self.state

    # Calculate new position
    if action == 0:    # Up
        new_state = (row - 1, col)
    elif action == 1:  # Down
        new_state = (row + 1, col)
    elif action == 2:  # Left
        new_state = (row, col - 1)
    else:              # Right
        new_state = (row, col + 1)

    # Check boundary
    if (new_state[0] < 0 or new_state[0] >= self.grid_size or
        new_state[1] < 0 or new_state[1] >= self.grid_size):

        reward = -5
        done = False

    # Check obstacle
    elif new_state in self.obstacles:

        reward = -5
        done = False

    else:
        self.state = new_state

        if self.state == self.goal:
            reward = 10
            done = True
        else:
            reward = -1
            done = False

    state_number = self.state[0] * self.grid_size + self.state[1]

    return state_number, reward, done, {}

# Create environment
env = GridWorldEnv()

state = env.reset()

print("Initial State:", state)

for i in range(10):
    action = env.action_space.sample()
    state, reward, done, info = env.step(action)

    print("State:", state,
          "Reward:", reward)

    if done:
        print("Goal Reached!")
        break

```

Conclusion

The Grid World problem demonstrates how Reinforcement Learning can solve navigation problems while satisfying constraints. The learned policy should guide the agent toward the goal while avoiding obstacles and unnecessary movements.

2. ϵ -Greedy Multi-Armed Bandit with 500 Trials

Aim

Implement an ϵ -greedy Multi-Armed Bandit algorithm and compare ϵ values of 0.1, 0.3, and 0.5 under a budget of 500 trials.

Concept

A Multi-Armed Bandit contains several actions, called arms. Each arm provides a reward with an unknown probability.

The ϵ -greedy strategy works as follows:

- With probability ϵ , choose a random arm (exploration).
- With probability $1 - \epsilon$, choose the arm with the highest estimated reward (exploitation).

Algorithm

1. Initialize estimated reward for each arm.
2. Select an arm using ϵ -greedy strategy.
3. Receive reward.
4. Update estimated reward.
5. Repeat for 500 trials.
6. Compare total rewards.

Python Implementation

```
import random

# True reward probabilities of 4 arms
true_probabilities = [0.2, 0.5, 0.7, 0.4]

# Number of trials
trials = 500

def run_bandit(epsilon):

    # Estimated values of each arm
    estimates = [0, 0, 0, 0]

    # Number of times each arm was selected
    counts = [0, 0, 0, 0]

    total_reward = 0
    exploration = 0
    exploitation = 0

    for i in range(trials):

        # Exploration
        if random.random() < epsilon:
            arm = random.randint(0, 3)
            exploration += 1

        # Exploitation
        else:
            arm = estimates.index(max(estimates))
            exploitation += 1
```

```

# Generate reward
if random.random() < true_probabilities[arm]:
    reward = 1
else:
    reward = 0

# Update count
counts[arm] += 1

# Update estimated reward
estimates[arm] += (
    reward - estimates[arm]
) / counts[arm]

total_reward += reward

return total_reward, exploration, exploitation

# Compare epsilon values
for epsilon in [0.1, 0.3, 0.5]:

    reward, explore, exploit = run_bandit(epsilon)

    print("\nEpsilon:", epsilon)
    print("Total Reward:", reward)
    print("Exploration:", explore)
    print("Exploitation:", exploit)

```

Analysis

- **$\epsilon = 0.1$:** The agent mostly exploits known information. It can achieve high rewards if it identifies the best arm quickly, but it may miss better arms.
- **$\epsilon = 0.3$:** Provides a reasonable balance between discovering good arms and using known good arms.
- **$\epsilon = 0.5$:** Performs much more exploration. It learns about different arms but spends many of the 500 trials trying potentially poor actions.

The exact rewards may vary because the program uses random numbers.

Constraint Satisfaction

The main constraint is the **500-trial budget**. The algorithm must maximize total reward without exceeding 500 trials.

Conclusion

For a limited budget, an intermediate ϵ value such as **0.3** can provide a good balance between exploration and exploitation. However, the optimal value depends on the reward distribution and environment.

3. MDP for an Autonomous Robot

Problem

Design an MDP for an autonomous robot that must reach a destination while minimizing energy consumption.

An MDP is represented as:

$$MDP = (S, A, P, R, \gamma)$$

where:

- **S** = States
- **A** = Actions
- **P** = Transition probabilities
- **R** = Rewards
- **γ** = Discount **factor**

1. States

The robot's state can include:

$$S = (\text{location}, \text{battery_level})$$

Example:

- Location: A, B, C, D, Goal
- Battery: High, Medium, Low

Example states:

- (A, High)
- (B, High)
- (C, Medium)
- (D, Low)
- (Goal, Medium)

2. Actions

The robot can perform:

- Move North
- Move South
- Move East
- Move West
- Stay
- Recharge

3. Transition Probabilities

The robot may not always move exactly as expected.

Example:

$$P(s' | sa) = 0.9$$

for the intended movement, and:

$$P(s' | sa) = 0.1$$

for an unexpected movement or failure.

For example, if the robot moves East:

- 0.9 → successfully moves East
- 0.1 → remains in the same location

4. Reward Function

A possible reward system is:

Event	Reward
Reach destination	+100
Move normally	-2
High energy consumption	-5
Low battery	-10
Collision	-50
Recharge	-1

The robot receives a high reward for reaching its destination and negative rewards for wasting energy.

A mathematical reward function can be:

$$\begin{aligned} R(s, a, s') &= 100 \text{ if goal is reached} \\ R(s, a, s') &= -2 - \text{EnergyCost}(a) \end{aligned}$$

for normal movements.

5. Energy Constraint

Let:

$$E_t = \text{current energy}$$

The robot must satisfy:

$$E_t \geq E_{min}$$

The robot cannot perform an action if its required energy is greater than the available energy.

Example

Suppose the robot starts at location A with 100 energy units.

- A → B costs 10 units
- B → C costs 15 units
- C → Goal costs 20 units

Total required energy:

$$10 + 15 + 20 = 45$$

Since the robot has 100 units, it can reach the goal without recharging.

Conclusion

The MDP enables the robot to choose actions based on both location and battery level. By assigning energy-related penalties, the RL agent learns a policy that reaches the destination while minimizing energy consumption

4. Value Iteration for a Constrained Grid Environment

Aim

Use Python and TensorFlow/Keras to implement Value Iteration and determine the optimal policy in a grid environment containing restricted states.

Bellman Optimality Equation

The value of a state is calculated using:

$$V(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | sa) V(s') \right]$$

For a deterministic environment:

$$V(s) = \max_a [R(s, a) + \gamma V(s')]$$

The optimal action is:

$$\pi(s) = \arg \max_a [R(s, a) + \gamma V(s')]$$

Grid

Consider the following grid:

```
S . . .  
. X . .  
. . X .  
. . . G
```

Python and TensorFlow/Keras Implementation

```
import numpy as np  
import tensorflow as tf  
from tensorflow import keras
```

```
# Grid size  
rows = 4  
cols = 4
```

```
# States  
start = (0, 0)
```

```

goal = (3, 3)

# Restricted states
restricted = [(1, 1), (2, 2)]

# Actions
# 0 = Up
# 1 = Down
# 2 = Left
# 3 = Right
actions = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1)
]

# Initialize value table
V = np.zeros((rows, cols))

# Discount factor
gamma = 0.9

# Number of iterations
iterations = 100

def get_next_state(state, action):

    row, col = state

    new_row = row + actions[action][0]
    new_col = col + actions[action][1]

    # Boundary constraint
    if new_row < 0 or new_row >= rows:
        return state

    if new_col < 0 or new_col >= cols:
        return state

    # Restricted state constraint
    if (new_row, new_col) in restricted:
        return state

    return (new_row, new_col)

# Value Iteration
for iteration in range(iterations):

    new_V = np.copy(V)

    for row in range(rows):
        for col in range(cols):

            state = (row, col)

            # Skip restricted states
            if state in restricted:
                continue

            # Goal value
            if state == goal:
                new_V[row, col] = 100

```

```

        continue

    action_values = []

    for action in range(4):

        next_state = get_next_state(state, action)

        # Reward
        if next_state == goal:
            reward = 100
        elif next_state == state:
            reward = -10
        else:
            reward = -1

        next_value = V[
            next_state[0],
            next_state[1]
        ]

        value = reward + gamma * next_value

        action_values.append(value)

    new_V[row, col] = max(action_values)

V = new_V

# Display value table
print("Optimal Value Table:")
print(V)

# Generate optimal policy
policy = np.empty((rows, cols), dtype=object)

symbols = ["U", "D", "L", "R"]

for row in range(rows):
    for col in range(cols):

        state = (row, col)

        if state in restricted:
            policy[row, col] = "X"

        elif state == goal:
            policy[row, col] = "G"

        else:

            action_values = []

            for action in range(4):

                next_state = get_next_state(state, action)

                if next_state == goal:
                    reward = 100
                elif next_state == state:
                    reward = -10
                else:
                    reward = -1

```

```

        value = reward + gamma * V[
            next_state[0],
            next_state[1]
        ]

        action_values.append(value)

    best_action = np.argmax(action_values)

    policy[row, col] = symbols[best_action]

print("\nOptimal Policy:")
print(policy)

```

Conclusion

Value Iteration systematically evaluates all possible actions and computes the optimal value of each state. The Bellman optimality equation allows the agent to select the best action while respecting restricted-state constraints.

5. RL Framework for a Warehouse Robot with a 30-Minute Battery

Problem

A warehouse robot has a battery capacity of only 30 minutes. It must complete the maximum possible number of deliveries without exceeding the battery limit.

The objective is:

max Number of Deliveries

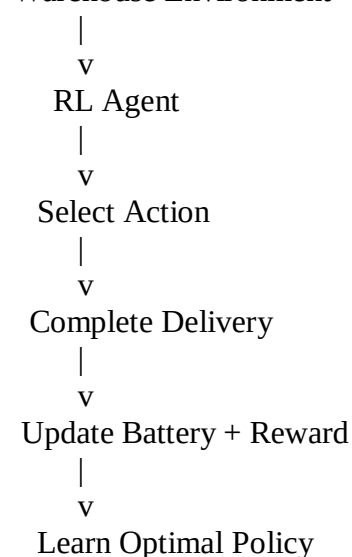
subject to:

***BatteryUsed* ≤ 30 minutes**

RL Framework

The RL system contains:

Warehouse Environment



1. State Space

The state can be represented as:

$S = (location, battery, pending_deliveries)$

For example:

State = (A, 25, 5)

means:

- Robot is at location A.
- 25 minutes of battery remain.
- 5 deliveries are pending.

The state may also include:

- Distance to next delivery
- Current delivery status
- Charging station availability

2. Action Space

The robot can choose:

- Move to delivery location
- Complete delivery
- Return to charging station
- Recharge
- Wait

3. Reward Function

A suitable reward function is:

Action/Event	Reward
Complete delivery	+20
Move toward delivery	-1
Use battery	-1 per minute
Return to charger	+2
Battery exhausted	-100
Complete maximum deliveries	+50

A simplified reward is:

$$R = 20D - E - P$$

where:

- D = number of completed deliveries
- E = energy/battery consumption
- P = penalty for violating constraints

4. Battery Constraint

The main constraint is:

$$BatteryRemaining \geq 0$$

and:

$$BatteryUsed \leq 30$$

Before selecting an action, the agent checks whether enough battery is available.

For example:

if `battery_remaining >= required_energy`:

`perform_action()`

else:

`return_to_charging_station()`

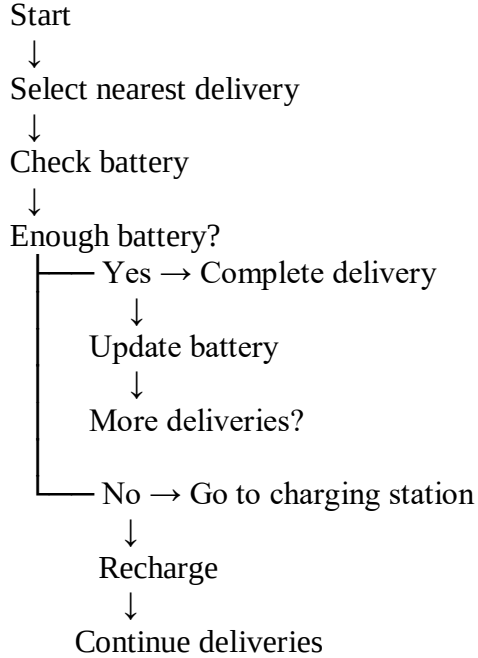
5. Policy

The policy should learn to:

1. Select nearby deliveries first.
2. Minimize unnecessary movement.
3. Avoid wasting battery.

4. Complete as many deliveries as possible.
5. Return to the charging station before the battery becomes empty.

A good policy may be:



6. Example

Suppose the robot has 30 minutes of battery.

	Delivery	Travel Time	Delivery Time	Total
D1	3 min	2 min	5 min	
D2	4 min	2 min	6 min	
D3	5 min	2 min	7 min	
D4	6 min	2 min	8 min	

If the robot completes D1, D2, and D3:

$$5 + 6 + 7 = 18 \text{ minutes}$$

The robot still has:

$$30 - 18 = 12 \text{ minutes}$$

The robot can then determine whether D4 can be completed safely. If not, it should return to the charging station.

Constraint-Based RL Objective

The optimization problem can be written as:

$$\max_{\pi} E \left[\sum_{t=0}^T R_t \right]$$

Subject to:

$$\sum_{t=0}^T \text{Energy}_t \leq 30$$

The agent learns a policy:

$$\pi(a | s)$$

that selects the best action based on the current location, battery level, and delivery status.

Conclusion

The RL framework allows the warehouse robot to dynamically select delivery routes based on battery availability. By giving high rewards for successful deliveries and strong penalties for battery exhaustion, the agent learns a policy that maximizes deliveries while satisfying the 30-minute battery constraint.